Claude Code - Quick Reference Guide

What is Claude Code?

Claude Code is an AI-powered command-line tool that acts as your coding assistant directly in the terminal. It can:

- Read and understand your entire codebase
- Write, edit, and refactor code
- Run tests and debug errors
- Execute git operations and manage workflows
- Generate documentation and tests

Installation

Claude Code on the web | Claude

macOS / Linux / WSL

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows

Use Windows Subsystem for Linux (WSL):

```
wsl --install
```

Then follow Linux installation steps inside WSL.

Complete Guide: Installing Claude Code on Windows 10 | Claude

Claude Code Installation Guide for Windows 11: Setting Up with WSL2 | Claude

Requirements

- Claude Pro subscription ($20/month) OR Anthropic API key
- Terminal access

Getting Started

First Run

```
# Navigate to your project directory
cd ~/your-project
```

```
3
4 # Start Claude Code
5 claude
```

First time: Browser opens for authentication. Login and grant permissions.

Basic Usage

```
claude                    # Start interactive session
claude "query"           # Start with initial prompt
claude -c                # Continue last conversation
claude -p "query"        # Print mode (non-interactive)
```

Writing Effective Prompts

✅ Good Prompts (Specific)

```
"Create a Flask REST API with POST /users endpoint that validates email
and password"

"Add error handling to the login() function in @auth.py"

"Refactor @database.py to use connection pooling for better
performance"

"Write pytest tests for all functions in @utils.py"
```

❌ Poor Prompts (Vague)

```
"Make an API"
"Fix this"
"Improve the code"
"Add features"
```

Best Practices

1. **Be specific** - Define what you want and how
2. **Reference files** - Use `@filename` to reference specific files
3. **Break it down** - Tackle one feature at a time
4. **Provide context** - Mention tech stack, requirements, constraints
5. **Ask for quality** - Request tests, error handling, documentation

---

Essential Commands

In-Session Commands (start with /)

Command	Description
`/init`	Create CLAUDE.md (project memory file)
`/clear`	Reset conversation context
`/help`	Show all available commands
`/rewind`	Undo last changes (or press ESC twice)

`/bug`	Report issues to Anthropic

File References

- `@filename.py` - Reference specific file in prompts
- `@folder/` - Reference entire folder

Control Keys

- `Ctrl+C` (twice) - Stop current action
- `Shift+Enter` - New line in prompt (after `/terminal-setup`)

CLAUDE.md - Project Memory

Create a `CLAUDE.md` file in your project root (use `/init`) to give Claude context:

```
# Project: Expense Tracker

## Architecture
- Backend: Python Flask REST API
- Database: SQLite
- Frontend: HTML/CSS/JS with Chart.js

## Coding Standards
- Follow PEP 8 for Python
- Use meaningful variable names
- Add docstrings to all functions
- Write tests for new features

## File Organization
- `/app.py` - Main Flask application
- `/database.py` - Database operations
- `/templates/` - HTML files
- `/static/` - CSS, JS, images
```

Why it matters: Claude reads this file at the start of every session, remembering your project structure and preferences.

Tips & Tricks

🚀 Speed Up Development

- **Skip permissions**: `claude --dangerously-skip-permissions` (use carefully!)
- **Custom commands**: Create `.claude/commands/test.md` → use `/test` anytime
- **Continue sessions**: `claude -c` picks up where you left off

🎯 For Better Results

- **Clear context regularly**: Use `/clear` between different tasks
- **Build incrementally**: Start simple, add features one by one
- **Ask Claude to explain**: "Explain how @auth.py works"
- **Request improvements**: "Make this more readable", "Add better error messages"

🛠️ Common Patterns

```
"Create [X] with features [A, B, C]"
"Add [feature] to @file.py"
"Fix the bug in @file.py on line 42"
"Write tests for @module.py using pytest"
"Refactor @file.py for better performance"
"Update README.md with setup instructions"
```

📦 Advanced Features

- **Subagents**: Create specialized AI assistants with `/agents`
- **Hooks**: Run scripts automatically (format on save, test before commit)
- **MCP**: Connect external tools (GitHub, Jira, etc.) with `claude mcp add`

Common Use Cases

✅ **Starting new projects** - Generate boilerplate and structure
✅ **Adding features** - Implement functionality quickly
✅ **Refactoring** - Clean up and optimize existing code
✅ **Writing tests** - Generate comprehensive test coverage
✅ **Debugging** - Find and fix errors
✅ **Documentation** - Create README, API docs, comments
✅ **Learning** - Understand unfamiliar codebases
✅ **Code review** - Analyze for bugs and improvements

Troubleshooting

Problem	Solution
"Command not found"	Restart terminal or check PATH
Context feels cluttered	Use `/clear` to reset
Made a mistake	Use `/rewind` or ESC twice
Need to stop Claude	Press Ctrl+C twice
Want previous session	Run `claude -c`

Resources

Documentation: Claude Code overview - Claude Docs
GitHub: GitHub - anthropics/claude-code: Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands.

Remember: Claude Code is a tool to augment your skills, not replace your judgment. Review generated code, ask questions, and iterate until it's exactly what you need.
